



Adaptive Recommendation Engine

using Coevolutionary Algorithms

Capital University (formerly Helwan University)
Faculty of Computing & Artificial Intelligence
Artificial Intelligence Department | Module: AI420

Team Members	Academic Details
Member 1: Ahmad Alaa	Semester: Spring 2026
Member 2 Yassin Yasser	Academic Year: 2025-2026
Member 3 Mahmoud Hossam	Submission: Week 13/14
Member 4 Nour Hatem	Dataset: MovieLens ML-100K
Member 5 Youssef Sami	Language: Python 3.11

1. Project Idea & Problem Formalisation

1.1 Overview

This project implements an Adaptive Recommendation Engine that employs Coevolutionary Algorithms (CoEAs) to learn latent-factor representations for collaborative filtering. Two sub-populations — one encoding User latent vectors and one encoding Item latent vectors — evolve simultaneously, each shaping the selection pressure on the other, in a manner analogous to host-parasite co-adaptation in biological ecosystems.

The engine is trained and evaluated on the MovieLens ML-100K dataset, a canonical benchmark containing 100,000 ratings (scale 1-5) from 943 users on 1,682 movies. The coevolutionary loop replaces gradient-based matrix factorisation (e.g., SGD or ALS), making the system robust to non-differentiable fitness landscapes and capable of escaping local optima that trap deterministic optimisers.

1.2 Problem Formalisation

The problem is formalised as a Constrained Real-Valued Optimisation problem:

- **Objective:** Minimise the Root Mean Square Error (RMSE) of predicted ratings across all observed (user, item) pairs.
- **Decision Variables:** Two real-valued matrices $U \in \mathbb{R}^{(n_u \times d)}$ and $V \in \mathbb{R}^{(n_i \times d)}$, where d is the latent dimension.
- **Constraint:** All latent factor values are bounded to the interval $[-3, 3]$, enforced via a repair (clip) function after every variation step.
- **Regularisation:** An L2 penalty term $\lambda(\|U\|^2 + \|V\|^2)$ is added to the fitness to discourage overfitting — a soft constraint encoded directly into the evaluation function.

Formally: minimise $f(U, V) = \text{RMSE}(U, V) + \lambda(\|U\|^2 + \|V\|^2)$ subject to $u_{ij}, v_{ij} \in [-3, 3]$ for all i, j .

2. Main Functionalities

The system is structured around a Streamlit web interface with three interactive tabs, each exposing a distinct functional layer of the coevolutionary engine:

2.1 Run Evolution Tab

- Executes a single coevolutionary run with the parameters configured in the sidebar.
- Displays real-time progress (generation counter + live RMSE) via a progress bar.
- Renders three post-run charts: RMSE convergence curve, best individual fitness per generation (User & Item populations separately), and population diversity dynamics.
- Shows a Top-5 recommendation matrix for the first 10 users derived from the evolved latent factor matrices.

2.2 Benchmarking Tab

- Runs an automated parameter sweep across all configurable operators (parent selection, crossover, mutation, survivor selection, representation, initialisation, diversity method).
- Executes each setting over up to 30 independent seeds, reporting mean RMSE, standard deviation, and best RMSE.
- Renders a 2x4 bar-chart grid with error bars for visual comparison.

- Exports all results to a structured JSON file for offline statistical analysis.

2.3 Educational Mode Tab

- Supports side-by-side (A/B) comparison of two fully independent algorithm configurations.
- Plots convergence, fitness landscape, and diversity dynamics for both configurations simultaneously.
- Designed as a teaching tool: students can vary one operator at a time and observe its direct effect on the evolutionary trajectory.

2.4 Sidebar Configuration Panel

- Unified control panel for all 18+ algorithm parameters: population size, latent dimension, generation count, coevolution mode, operator choices, regularisation strength, PSO hybrid toggle, adaptive mutation toggle, over-selection toggle.
- Supports two dataset sources: local ML-100K path or direct file upload (CSV/TSV/TXT).

3. Similar Applications in the Market

System	Approach	Key Difference vs. Ours
Netflix Cinematch / MF	SGD / ALS Matrix Factorisation	Gradient-based; cannot handle non-differentiable fitness
Spotify Collaborative Filter	Neural Collaborative Filtering (NCF)	Requires large labelled data + GPU training
Amazon Item-to-Item CF	Item-based cosine similarity	Static; no evolutionary adaptation
EvoRec (research)	Single-population GA on user profiles	No co-adaptation between User & Item populations
Our System	Dual-population CoEA + hybrid PSO + EA operators	Fully evolutionary; no gradients; interpretable parameters

4. Literature Review

The following six papers directly underpin the theoretical and practical decisions made in this project:

Paper 1

Potter, M. A., & De Jong, K. A. (1994). "A cooperative coevolutionary approach to function optimisation." In Proc. PPSN III, LNCS 866, pp. 249-257. Springer.

Foundational paper establishing cooperative coevolution (CCEA) for continuous optimisation. The dual-population architecture of our engine — where User and Item sub-populations co-evolve and are evaluated using representatives from the opposing population — directly follows Potter & De Jong's collaborator-selection framework.

Paper 2

Rosin, C. D., & Belew, R. K. (1997). *"New methods for competitive coevolution."* Evolutionary Computation, 5(1), 1-29.

Introduces competitive coevolutionary dynamics and the concept of relative fitness. Our competitive mode, where Item fitness is negated to simulate adversarial pressure against the User population, is a direct application of the host-parasite co-adaptation model described in this work.

Paper 3

Storn, R., & Price, K. (1997). *"Differential evolution — a simple and efficient heuristic for global optimization over continuous spaces."* Journal of Global Optimization, 11(4), 341-359.

The seminal DE paper. We implement DE/rand/1/bin as a crossover operator (cx_de) within the coevolutionary loop, allowing the engine to exploit vector-difference mutation alongside BLX-alpha and uniform crossover. This hybrid use of DE as a recombination primitive — rather than a standalone algorithm — follows the direction suggested by Storn & Price for embedding DE in larger evolutionary frameworks.

Paper 4

Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). *"Item-based collaborative filtering recommendation algorithms."* In Proc. WWW 2001, pp. 285-295. ACM.

Establishes the theoretical basis for item-based collaborative filtering and introduces the MovieLens dataset as the standard benchmark for recommendation research — the same dataset (ML-100K) used in our experiments. The RMSE evaluation metric and the train/test split strategy are directly inherited from this work.

Paper 5

Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). *"A fast and elitist multiobjective genetic algorithm: NSGA-II."* IEEE Trans. Evolutionary Computation, 6(2), 182-197.

Introduces crowding distance as a diversity-preservation mechanism — one of the two diversity strategies implemented in our engine (alongside fitness sharing). The crowding-distance-based fitness adjustment in our `surv_select` function adapts the NSGA-II crowding operator to a single-objective setting to maintain spread across the Pareto front of the latent factor space.

Paper 6

Derrac, J., Garcia, S., Molina, D., & Herrera, F. (2011). *"A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms."* Swarm and Evolutionary Computation, 1(1), 3-18.

Provides the statistical methodology underpinning our benchmarking design. The requirement for 30 independent runs per setting, the use of fixed seeds (provided in `seeds.json`), and the reporting of mean $\pm$ std RMSE are all mandated by the Wilcoxon signed-rank test protocol described in this paper. Our benchmark module's results table is designed to be directly importable into such a statistical comparison.

5. Dataset

MovieLens ML-100K

Source: GroupLens Research — <https://grouplens.org/datasets/movielens/100k/>

The ML-100K dataset is the canonical benchmark for collaborative filtering research, collected by the GroupLens lab at the University of Minnesota and released in 1998. It remains the standard entry-point dataset for recommendation system evaluation due to its well-studied statistical properties and clean, publicly available format.

Property	Value	Notes
Total Ratings	100,000	Integer scale 1–5
Users	943	Anonymised IDs 1–943
Items (Movies)	1,682	With genre metadata
Sparsity	~93.7%	Typical for CF benchmarks
File Format	TSV (u.data)	user_id item_id rating timestamp

IDs are converted from 1-indexed (native ML-100K format) to 0-indexed during loading to align with NumPy array addressing. Ratings are stored as float32 for efficient vectorised RMSE computation. The dataset is loaded at runtime via `load_ml100k()` or can be substituted with any user-uploaded TSV file via `parse_upload()`.

6. Algorithm Details & Experimental Results

6.1 EA Component Definitions

EA Component	Implementation
Representation	Real-valued matrices (default) or sign-binarised matrices. Each individual = one ($n_entities \times latent_dim$) float32 array.
Population	Two independent populations of size <code>pop_size</code> (default 20): one for Users, one for Items.
Fitness Function	$f(ind) = RMSE(predicted, actual) + \lambda \cdot mean(ind^2)$. Evaluated using the best individual from the opposing population as representative.
Initialisation	Random: $U[-3, 3]$ uniform. Heuristic: first latent dimension seeded from $mean(rating - 3) / 2$ for entities with observed ratings.
Parent Selection	Tournament ($k=3$) or Roulette Wheel (inverse-fitness proportional). Over-selection available as bonus mode.
Crossover	Uniform (50% gene swap), BLX-alpha (blend in $[lo-\alpha \cdot d, hi+\alpha \cdot d]$, default $\alpha=0.5$), DE/rand/1/bin ($F=0.8$, $CR=cx_prob$).
Mutation	Gaussian (additive noise, σ adaptive) or Polynomial (Deb's SBM, $\eta=20$). Adaptive σ self-adapts via log-normal rule each generation.
Survivor Selection	Elitism ($\mu+\lambda$): merge parents + offspring, keep best <code>pop_size</code> . (μ,λ): offspring-only replacement with $\lambda=2 \cdot \mu$.
Diversity	Fitness Sharing ($\sigma_share=2.0$, $\alpha=1.0$) or Crowding Distance (NSGA-II-style, adapted to single-objective).
Constraint Handling	Repair function: <code>np.clip(individual, -3, 3)</code> applied after every mutation step.
Termination	Fixed generation budget (default 100; configurable up to 500 via sidebar).

Hybrid PSO (bonus)	Optional PSO velocity update applied to the surviving population each generation, guided by personal best and global best latent vectors.
--------------------	---

6.2 Coevolutionary Modes

Cooperative: Both populations minimise RMSE jointly. Item fitness = $\text{RMSE} + \lambda \|V\|^2$. Sub-populations improve together — analogous to mutualism.

Competitive: Item fitness is negated ($i_fit = -\text{RMSE}$). Items 'try' to maximise User prediction error, forcing Users to evolve more robust latent factors — analogous to host-parasite dynamics.

6.3 Parameter Settings

Parameter	Default Value	Range Explored
Population Size	20	5 – 100
Latent Dimension	8	2 – 32
Generations	100	10 – 500
Mutation Rate	0.15	0.01 – 0.50
Crossover Prob.	0.80	0.10 – 1.00
Regularisation λ	0.01	0.00 – 0.10
BLX Alpha	0.50	Fixed
DE F factor	0.80	Fixed
Tournament k	3	Fixed
Seeds	1 – 30 (seeds.json)	30 independent runs

6.4 Benchmark Results Summary

All experiments are executed via the Benchmarking Tab using the automated `run_benchmarks()` function. Each configuration is run over `n_runs` independent seeds (up to 30) drawn from the `seeds.json` file. Results are reported as Mean RMSE $\pm$ Std across runs.

Parameter	Setting A	RMSE (A)	Setting B	RMSE (B)
Coevo Mode	Cooperative	5.3617	Competitive	5.8711
Parent Selection	Tournament	5.3617	Roulette	5.8711
Crossover	BLX-Alpha	5.3617	Uniform	5.8711
Mutation	Gaussian	5.3617	Polynomial	5.8711
Survivor	Elitism ($\mu+\lambda$)	5.3617	(μ,λ)	5.8711
Diversity	Fitness Sharing	5.3617	Crowding	5.8711
Representation	Real-valued	5.3617	Binary	5.8711
Initialisation	Random	5.3617	Heuristic	5.8711

7. Development Platform

Component	Tool / Version	Role
Language	Python 3.11	Primary implementation language
UI Framework	Streamlit 1.x	Interactive web interface
Numerical Engine	NumPy 1.26	Vectorised EA operations & PRNG
Data Handling	Pandas 2.x	Dataset loading & result tables
Visualisation	Matplotlib 3.x	Convergence curves & benchmark plots
Dataset	MovieLens ML-100K	GroupLens Research — public domain
OS	Windows 10 / Ubuntu 22.04	Cross-platform compatible
Seed File	seeds.json (provided)	30 fixed seeds for reproducibility
Version Control	Git / GitHub	Team collaboration & submission

8. Team Member Contributions

Member	Primary Role	Specific Contributions
Ahmad Alaa	Algorithm Lead	Coevolutionary engine, fitness function, constraint repair
Yassin Yasser	Operator Implementation	Crossover operators (BLX- α , Uniform, DE), mutation operators
Mahmoud Hossam	Diversity & Selection	Fitness sharing, crowding distance, tournament & roulette selection
Nour Hatem	UI & Visualisation	Streamlit interface (all 3 tabs), Matplotlib charts, benchmarking pipeline
Youssef Sami	Data & Experiments	ML-100K loader, benchmarking runs, seeds.json, statistical analysis

9. Assessment Criteria Compliance Checklist

Requirement	Status
Coevolutionary algorithm (two sub-populations: Users & Items)	✓ Done
Cooperative AND competitive coevolution modes	✓ Done
Clearly defined fitness function (RMSE + L2 regularisation)	✓ Done
Problem formalised as constrained real-valued optimisation	✓ Done
Constraint handling via repair function (clip to [-3, 3])	✓ Done
≥2 parent selection methods (Tournament & Roulette)	✓ Done
≥2 crossover operators (Uniform, BLX-alpha, DE)	✓ Done
≥2 mutation operators (Gaussian, Polynomial)	✓ Done
≥2 survivor selection methods (Elitism $\mu+\lambda$, μ -lambda) (bonus)	✓ Done
≥2 representations (Real-valued, Binary) (bonus)	✓ Done
≥2 initialisation methods (Random, Heuristic) (bonus)	✓ Done
Diversity preservation: Fitness Sharing + Crowding Distance	✓ Done
Parameter control: adaptive mutation sigma (self-adaptation)	✓ Done
Over-selection for large populations (bonus)	✓ Done
Hybrid approach — PSO integrated as optional post-selection step (bonus)	✓ Done
30 fixed seeds stored and provided (seeds.json)	✓ Done
Functional user interface demonstrating parameters, inputs, results	✓ Done
Educational visual interface for teaching effects of parameters (bonus)	✓ Done
Publicly available dataset (MovieLens ML-100K)	✓ Done
At least 4–6 academic literature papers reviewed	✓ Done (6 papers)
Documentation report with all 7 required sections	✓ This document